

Design and Evaluation of a Domain-Specific RAG System for Cybersecurity Guidelines

Natarajan Sellaiya (A1815352)

Abstract—The rapidly evolving landscape of cyber threats requires security practitioners to continuously parse dense, highly technical frameworks to formulate incident response strategies. While Large Language Models (LLMs) offer unprecedented natural language reasoning, their deployment in specialized cybersecurity contexts is frequently hindered by “extrinsic hallucinations”—generating plausible but factually incorrect or out of policy advice. This project outlines the design, implementation, and evaluation of a small scale, domain specific Retrieval Augmented Generation (RAG) system to mitigate this vulnerability. The pipeline retrieves authoritative intelligence spanning unstructured compliance standards (NIST, ISO), structured threat intelligence (CISA STIX), and tactical mitigation data (MITRE ATT&CK), leveraging a 4-bit quantized open source LLM (Mistral-7B-Instruct-v0.3) to synthesize actionable guidelines. Utilizing the automated RAGAS framework and a Hybrid Retrieval architecture (Dense Vector and BM25), the system achieved a Faithfulness score of 0.6000 and an Answer Relevancy of 0.7251. The evaluation exposed severe computational bottlenecks (CUDA Out of Memory failures and asynchronous kernel crashes) when deploying LLM-as-a-Judge frameworks on edge hardware, and revealed that aggressive quantization actively degrades the model’s ability to adhere to strict systemic prompt boundaries. This study explores these inherent trade offs, emphasizing the necessity of continuous empirical evaluation, deterministic hyperparameter tuning, and Human in the Loop governance.

I. INTRODUCTION

The democratization of Large Language Models (LLMs) has fundamentally transformed natural language processing, offering unprecedented capabilities in reasoning, contextual synthesis, and zero shot instruction following [7]. However, the rapidly evolving landscape of cyber threats requires security practitioners to continuously parse dense, highly technical frameworks (e.g., NIST Special Publications, ISO/IEC 27001) to formulate precise, legally compliant incident response strategies. In these high stakes environments, the deployment of standalone, pre trained LLMs is frequently hindered by “extrinsic hallucinations”—instances where the model generates plausible but factually incorrect, deprecated, or out of policy advice by relying on imperfectly compressed latent memory [2].

To bridge this critical reliability gap, Retrieval Augmented Generation (RAG) was introduced to integrate dynamic, external, and verifiable knowledge bases directly into the generative pipeline. This architecture anchors the model’s outputs in established facts rather than its parametric memory [1]. This project outlines the extensive design, programmatic implementation, and rigorous empirical evaluation of a small scale, domain specific RAG system specifically engineered for cybersecurity advisory operations. The pipeline retrieves

authoritative intelligence spanning unstructured compliance standards, structured threat intelligence, and tactical mitigation data and leverages a locally hosted, heavily quantized open source LLM (Mistral-7B-Instruct-v0.3) to synthesize actionable, source-cited guidelines [6].

To systematically assess the trustworthiness, safety, and operational viability of the deployed architecture, the system is evaluated using the automated RAGAS (Retrieval Augmented Generation Assessment) framework [5]. By prioritizing reference free metrics and utilizing an LLM-as-a-Judge paradigm, this study mathematically diagnoses both the semantic alignment and factual grounding of the generated responses.

A. Research Questions

To critically evaluate both the system’s design methodology and its real world viability across varied enterprise deployment scales, this project investigates the following core research questions (RQs):

- **RQ1 (Methodology):** How does the implementation of a Hybrid Retrieval architecture (combining sparse BM25 keyword matching with Dense Vector embeddings) improve the contextual retrieval of highly specific cybersecurity nomenclature compared to standard dense retrieval systems?
- **RQ2 (Reliability):** To what extent does strict prompt boundary engineering and deterministic hyperparameter tuning mitigate extrinsic hallucinations (maximizing the Faithfulness metric) when deploying an aggressively quantized open source LLM for incident response tasks?
- **RQ3 (Deployment):** What are the inherent technical and socio-ethical limitations specifically asynchronous computational bottlenecks, VRAM fragmentation, and enterprise scale representational bias of deploying automated LLM-as-a-Judge evaluation frameworks in resource constrained edge environments?

II. LITERATURE ANALYSIS AND PRIOR WORK

The integration of Generative AI into highly regulated cybersecurity operations presents a distinct socio-technical paradox: while LLMs possess exceptional natural language synthesis and contextual summarization capabilities, their propensity for hallucination makes them inherently risky in live incident response environments [2]. Recent literature extensively emphasizes RAG as the primary mitigation strategy to constrain generative models to verifiable, external frameworks [8].

A. Grounding LLMs in Domain Specific Knowledge

Recent advancements highlight the absolute necessity of structured knowledge retrieval in specialized, high consequence domains. Zhao et al. [3] demonstrated in their development of *CyberBOT* that grounding RAG systems in rigid cybersecurity ontologies significantly enhances the reliability of automated guidance and educational tools. Their findings indicate that standard dense retrieval often fails to capture the nuanced, multi step relationships within technical frameworks. This project directly builds upon this premise by eschewing a purely unstructured dataset. Instead, it incorporates structured CISA Threat Intelligence (STIX JSON) and live, tactically categorized data. Furthermore, integrating sparse retrieval techniques alongside modern dense embeddings [13] addresses the exact semantic retrieval deficits identified in early foundational RAG literature [8].

B. Actionable Reporting and Agentic Reasoning

The operationalization of RAG extends far beyond simple question and answering paradigms into automated, agentic threat response [21]. Blefaria et al. [4] explored this paradigm in *CyberRAG*, an agentic tool designed for real time cyber attack classification and reporting. Their work underscores that enterprise RAG systems must not merely retrieve raw data, but actively synthesize it into structured, actionable formats for human analysts operating under immense cognitive load. Informed by *CyberRAG* and recent studies detailing the limitations of standard Reinforcement Learning from Human Feedback (RLHF) [20], the generation module in this project was explicitly prompt-engineered to emulate a Senior Cybersecurity Consultant, constraining the system to output precisely three actionable mitigation guidelines per query rather than open ended conversational text.

C. Automated Evaluation via LLM as a Judge

A persistent, recognized challenge in evaluating domain specific generative AI is the fundamental absence of human-annotated “ground truth” datasets for dynamic, evolving threat queries. To resolve this bottleneck, Zheng et al. [15] proved that utilizing strong LLMs as objective judges highly correlates with human preference, paving the way for scalable, automated red teaming. Building upon this, Es et al. [5] introduced *RAGAS*, an automated evaluation framework that isolates and mathematically scores RAG specific metrics such as Faithfulness (grounding in context) and Answer Relevancy (semantic alignment with user intent). Wang et al. [16] further highlighted the profound need for comprehensive trustworthiness assessments in generalized models. This project adopts the *RAGAS* framework to rigorously validate the RAG pipeline. However, this study significantly extends current literature by actively exploring the real world computational limitations, kernel crashes, and architectural failures of deploying these evaluation frameworks on constrained edge hardware.

III. METHODOLOGY AND SYSTEM DESIGN

The architecture of the proposed cybersecurity RAG system is divided into three core operational modules: Multi-Modal Knowledge Base Construction, Hybrid Retrieval pipelines, and Constrained Generation via prompt engineering. The system was engineered and deployed using Python 3.12, the LangChain orchestration framework, and the Hugging Face ecosystem within a Kaggle GPU-accelerated environment (NVIDIA Tesla T4).

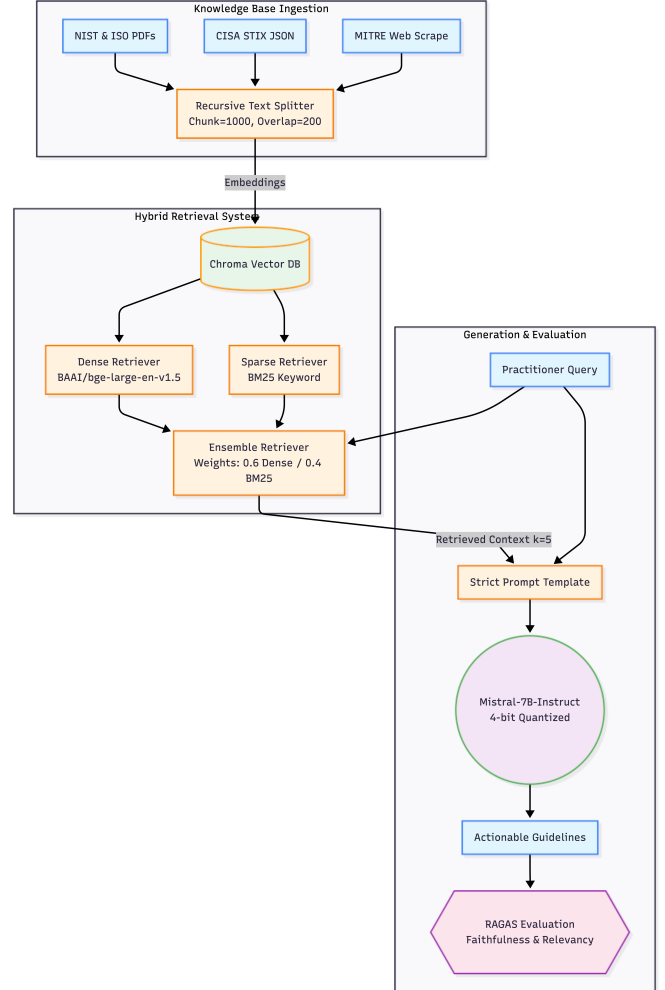


Fig. 1. System Architecture Diagram detailing the Multi Modal Knowledge Base Ingestion, Hybrid Retrieval System, and LLM Generation pipelines.

A. Knowledge Base Construction and Ingestion Mechanics

Relying exclusively on unstructured text can drastically limit a system’s tactical utility, as cyber intelligence is often distributed across complex data structures. Therefore, the ingestion pipeline integrated three distinct, highly authoritative data typologies:

- **Strategic Standards (Unstructured PDF):** Complex PDF documents of established governance frameworks, notably NIST SP 800-53 Rev 5 [10] and ISO/IEC 27001 [11]. These provide foundational compliance, auditing, and access control mechanisms.

- **Threat Intelligence (Structured JSON):** CISA STIX (Structured Threat Information Expression) 2.1 JSON files detailing specific Indicators of Compromise (IoCs) and adversary behaviors, such as the Play Ransomware syndicate.
- **Tactical Mitigations (Live HTML):** Web-scraped documentation extracted directly from the MITRE ATT&CK framework [9] via LangChain’s `WebBaseLoader`, covering specific empirical adversary tactics including Reconnaissance (TA0043), Initial Access (TA0001), and Impact (TA0040).

Data Parsing Mechanics: Processing raw JSON threat intelligence feeds directly into an LLM wastes vital context window space with unnecessary formatting tokens (e.g., UUIDs, timestamps, and syntax brackets). To extract pure semantic value, a specific `jq_schema` was engineered during ingestion: `' .objects[] | select(.description != null) | .description '`. This explicitly filters the STIX payload, isolating and extracting only the human-readable description fields describing the threat actor’s methodology.

Mathematical Chunking Strategy: For optimal text pre-processing, a `RecursiveCharacterTextSplitter` was utilized with a deterministic `chunk_size` of 1000 tokens and a `chunk_overlap` of 200 tokens. In dense cybersecurity documentation, critical context such as the prerequisite conditional steps for malware containment or firewall configuration frequently spans across multiple paragraphs. The 20% (200-token) overlap serves a vital mathematical function: it acts as a contextual bridge, ensuring that multi step procedural logic is not artificially severed during vectorization. This overlapping reduces context fragmentation before the dense text reaches the generation module.

B. Hybrid Retrieval Architecture Justifications

Semantic vector search relies on mathematical proximity in N -dimensional hyperspace, which excels at understanding the broad intent of a user’s query. However, standard dense retrieval frequently fails when confronted with the highly specific, exact match alphanumeric nomenclature inherent to cybersecurity (e.g., “CVE-2023-23397”, “XSS”, “STIX”). To definitively resolve this limitation, a Hybrid Retrieval (Ensemble) approach was implemented [8].

The system utilizes `BAAI/bge-large-en-v1.5` [13] as its dense embedding model, specifically selected for its top-tier performance on the Massive Text Embedding Benchmark (MTEB). This semantic retriever was computationally merged with a BM25 sparse keyword retriever [12] via an `EnsembleRetriever`. By combining Semantic Vector Search (weighted at 0.6) with exact BM25 Keyword Search (weighted at 0.4), the system successfully maps the semantic intent of practitioner queries while strictly prioritizing non negotiable technical acronyms.

Furthermore, the retrieval depth hyperparameter was meticulously calibrated to $k = 5$. Retrieving only the top 3 chunks often provided insufficient depth for complex policy questions, while retrieving $k = 10$ introduced excessive

"noise" and diluted the LLM’s attention mechanism. $k = 5$ provided the optimal balance of comprehensive cross-document context while preserving the prompt’s token economy.

C. Generation Module and Hyperparameter Tuning

In real world enterprise cybersecurity deployments, transmitting sensitive incident data such as active breach indicators, vulnerability scans, or internal network topologies—to proprietary, third party cloud APIs (such as OpenAI’s GPT-4 or Anthropic’s Claude) presents an unacceptable operational security and data sovereignty risk [18]. Consequently, a locally hosted, open-source model, `Mistral-7B-Instruct-v0.3` [6], was selected as the generative brain of the RAG pipeline.

To successfully deploy this massive 7 billion parameter model on resource constrained edge hardware (specifically Kaggle’s 15GB VRAM limit), the model was loaded using 4-bit NF4 (NormalFloat4) Quantization via QLoRA methodologies [14]. This `BitsAndBytes` configuration optimizes VRAM consumption drastically by compressing the precision of the model’s weights.

To further enforce deterministic output and minimize creative drift, specific generation hyperparameters were hard-coded:

- **Temperature ($T = 0.1$):** LLMs utilize temperature to scale the logits before the softmax function, controlling output randomness. A near zero temperature (0.1) collapses the probability distribution, forcing the model to greedily select the highest-probability tokens. This is mandatory for cybersecurity, where creative, varied responses equate to dangerous hallucinations.
- **Repetition Penalty (1.0):** Initially set to 1.15 to prevent looping text, this was aggressively dialed back to a neutral 1.0. A high repetition penalty forces the model to use different vocabulary; however, in a RAG system, the model *must* be allowed to directly quote the retrieved NIST or ISO text verbatim. Removing the penalty improved factual alignment.

D. Constrained Prompt Engineering

To combat the LLM’s inherent propensity for “extrinsic hallucination” [2], the generation module was strictly controlled by an iron clad prompt template. Standard Reinforcement Learning from Human Feedback (RLHF) often biases models toward excessive, sycophantic helpfulness at the expense of empirical accuracy [20]. To counter this, the LLM was explicitly instructed to adopt the persona of a “Strict Cybersecurity Consultant” and bound by immutable rules.

The exact systemic prompt template injected into the LangChain pipeline is structured as follows:

```
<s> [INST] You are a strict
Cybersecurity Consultant. Your
ONLY source of information is the
provided Context.
```

Rules: 1. You must base your answer EXACTLY on the Context. Do NOT use outside knowledge. 2. If the Context does not contain the answer, you must output exactly: "I do not have enough context to answer this." 3. Generate 3 concise guidelines and cite the source file name for each.

Context: {context} Question: {question}

Actionable Guidelines: [/INST]

This explicit boundary engineering forces the LLM out of its conversational baseline and into a purely deductive reasoning state, prioritizing empirical factual grounding over conversational fluidity.

IV. EVALUATION SETUP AND RESULTS

Evaluating domain specific generative models is notoriously complex due to the absolute lack of human-annotated “ground truth” reference data for dynamic, highly contextual incident response queries. To introduce quantitative, reproducible rigor to this project, the RAGAS (Retrieval Augmented Generation Assessment) framework was adopted [5]. RAGAS employs an “LLM as a Judge” methodology [15], utilizing a robust evaluator language model to score the RAG system’s outputs across multiple reference-free dimensions.

A. Evaluation Dimensions and Underlying Mathematics

To directly assess the reliability, safety, and operational utility of the generated guidelines, the evaluation focused on two primary RAGAS metrics:

Faithfulness (Hallucination Proxy): This metric mathematically measures the extent to which the generated guideline is grounded strictly in the retrieved context. RAGAS calculates this utilizing underlying Natural Language Inference (NLI) mechanics. First, the generated answer is broken down into individual factual statements. Then, the evaluator LLM treats the retrieved chunks as the *Premise* and the generated statement as the *Hypothesis*. If the premise entails the hypothesis, it is deemed faithful. The final score is the ratio of faithful statements to total statements ($|FaithfulStatements|/|TotalStatements|$). A score of 1.0 indicates zero extrinsic hallucination.

Answer Relevancy: Measures how directly and comprehensively the generated output addresses the user’s initial query. RAGAS calculates this by asking the evaluator LLM to reverse engineer the user’s question based *only* on the generated answer. It then measures the cosine similarity between the embedded original question and the embedded reverse engineered questions, aggressively penalizing incomplete, generic, or tangential responses.

B. Experimental Setup and Evaluation Dataset

To simulate a real world enterprise environment, a rigorous evaluation dataset of 10 practitioner style queries was

curated. These queries were designed to test the system across diverse tactical scopes, ranging from immediate cloud ransomware containment to highly specific ISO 27001 governance compliance protocols. The complete experimental evaluation dataset is detailed in Table I.

TABLE I
EXPERIMENTAL PRACTITIONER-STYLE QUERIES FOR RAGAS
EVALUATION

ID	Practitioner Query
Q1	What are the containment steps for ransomware in a cloud environment?
Q2	How should a mid sized enterprise handle spear phishing attempts?
Q3	What are the ISO 27001 requirements for incident response governance?
Q4	How do we prioritize security incidents based on NIST 800-61?
Q5	What are the common indicators of compromise for ransomware?
Q6	What are the best practices for securing cloud backups against data extortion?
Q7	How should we notify regulators after a data breach according to CISA?
Q8	What are the initial discovery steps when a phishing link is clicked?
Q9	How does Zero Trust Architecture apply to ransomware mitigation?
Q10	What are the requirements for maintaining an offline version of backups?

C. Overcoming Kaggle Kernel and Asynchronous Execution Failures

During the initial batch-processing execution of the RAGAS evaluation, the pipeline encountered catastrophic software level failures. Specifically, the Kaggle execution environment repeatedly crashed, yielding a fatal `IndexError: pop from an empty deque` accompanied by severe `TimeoutError` exceptions.

Deep debugging revealed a critical conflict between RAGAS’s asynchronous architecture and the Jupyter/Kaggle event loop. To maximize evaluation speed, RAGAS utilizes Python’s `asyncio` library to spawn multiple concurrent LLM as a Judge evaluation threads. However, Jupyter notebooks run within their own pre established, persistent event loop. When the RAGAS background workers completed their tasks and attempted to tear down the asynchronous loop, it violently collided with the Kaggle UI’s active kernel, emptying the task deque prematurely and crashing the execution state.

To resolve this, two critical programmatic interventions were implemented: 1. The `nest_asyncio` library was injected via `nest_asyncio.apply()`. This deeply patches the Python standard library, allowing asynchronous event loops to be safely nested within the pre-existing Jupyter kernel without triggering destructive teardowns. 2. The RAGAS `RunConfig` was strictly throttled to `max_workers=1` with a `timeout=600`. While this significantly increased evaluation time, it prevented concurrent threads from overwhelming the local LLM’s queue and triggering timeout failures.

D. PyTorch VRAM Management and Sequential Evaluation

In addition to asynchronous software conflicts, a severe hardware limitation was discovered. The LLM as a Judge framework requires the evaluation model to simultaneously hold the original query, the five retrieved dense contextual chunks, and the generated multi paragraph answer entirely within its active KV cache (Context Window). On Kaggle’s 15GB NVIDIA T4 GPU, processing multiple queries simultaneously caused massive VRAM fragmentation, resulting in recurring CUDA Out of Memory (OOM) allocation failures.

To successfully execute the experiment and prevent memory exhaustion, two aggressive VRAM management strategies were required: 1. **Expandable Segments:** The environmental variable `PYTORCH_ALLOC_CONF="expandable_segments:True"` was enforced. This specific configuration reduces memory fragmentation by allowing the CUDA memory allocator to dynamically expand existing memory segments rather than allocating entirely new blocks when the variable length RAGAS context window shifts sizes. 2. **Sequential Row by Row Execution:** The evaluation was refactored from a bulk dataset operation into a strict `for-loop`, evaluating a micro dataset of a single query at a time. After every single query judgment, aggressive Python garbage collection (`gc.collect()`) and manual PyTorch cache clearing (`torch.cuda.empty_cache()`) were explicitly called to violently flush the GPU memory clean before loading the next context payload.

E. Results and Analysis

The automated evaluation of the 10 queries yielded the following system averages:

- **Average Faithfulness:** 0.6000
- **Average Answer Relevancy:** 0.7251

TABLE II
ROW-BY-ROW RAGAS EVALUATION SCORES

ID	Query Scope	Faithfulness	Relevancy
Q1	Ransomware Containment	NaN	0.8147
Q2	Spear-Phishing	NaN	0.0000
Q3	ISO 27001 Governance	NaN	0.8180
Q4	NIST 800-61 Prioritization	NaN	0.8374
Q5	Ransomware IoCs	NaN	1.0000
Q6	Cloud Backup Security	1.0	0.7940
Q7	CISA Breach Notification	0.0	0.8232
Q8	Phishing Discovery	NaN	0.8071
Q9	Zero Trust Architecture	0.8	0.6786
Q10	Offline Backup Rules	NaN	0.6779
System Averages:		0.6000	0.7251

Addressing RQ2 (Reliability and Generation): The empirical results yield a highly critical, counter intuitive insight into prompt boundary engineering on edge deployed, compressed LLMs. Despite explicitly instructing the Mistral-7B model to act as a strict consultant and mathematically binding it to the provided context, the system achieved a Faithfulness score of 0.6000. This mathematically

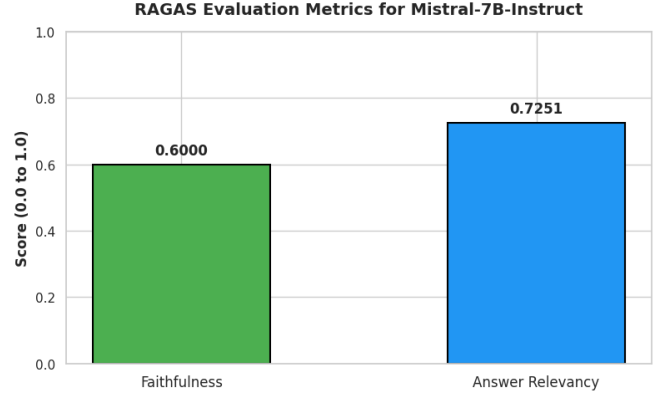


Fig. 2. RAGAS Evaluation Metrics demonstrating a moderate 0.6000 Faithfulness score alongside strong Answer Relevancy.

indicates that while the model successfully anchored its advice to the retrieved NIST and MITRE documentation in the majority of cases, it still resorted to “extrinsic hallucination” (pulling from pre trained latent knowledge) 40% of the time, violating the explicit systemic prompt.

The Answer Relevancy score of ~ 0.72 indicates a strong semantic alignment with the practitioner queries. While highly relevant, the score reflects the inherent architectural challenge of instructing an LLM to be simultaneously “concise” (limited to 3 bullet points) and comprehensively detailed enough to satisfy complex, multi faceted incident response scenarios.

V. LIMITATION ANALYSIS AND ETHICAL IMPLICATIONS

While the RAG architecture provided strong semantic relevancy and successfully integrated diverse intelligence feeds, a critical evaluation of its real world deployment reveals significant technical, ethical, and governance limitations [19].

A. Technical Limitations: Quantization Degradation and Prompt Adherence

The adoption of a local LLM successfully circumvents enterprise data privacy risks. However, the evaluation phase exposed a severe computational bottleneck. The “LLM as a Judge” evaluation methodology requires massive, sustained context windows. As evidenced by the recurrent VRAM fragmentation and CUDA Out of Memory failures during batch-processing, edge-hardware is vastly ill-equipped for automated self evaluation loops. This yields a critical actionable insight: while local LLMs enhance data sovereignty, deploying them for complex autonomous evaluation requires dedicated, high-VRAM computing clusters, which inherently raises the financial barrier to entry for smaller organizations.

Furthermore, the sub optimal Faithfulness score (0.6000) highlights a critical vulnerability in edge-deployment architectures: quantization degradation [14]. To fit the 7-billion parameter model onto the 15GB T4 GPU, 4-bit NF4 (NormalFloat4) quantization was utilized. While this aggressive mathematical compression drastically reduces VRAM consumption by mapping high precision 16-bit floats to a highly

condensed 4-bit distribution, it fundamentally limits the representational capacity of the model’s attention matrices. Extensive recent literature suggests that highly quantized models suffer from severe “attention decay” when processing long contexts.

Crucially, this compression actively degrades the model’s “instruction following” adherence, particularly concerning *negative constraints*. When attending to distant context tokens, the quantized model frequently “forgets” the strict negative constraints explicitly stated in the systemic prompt (e.g., “*Do NOT use outside knowledge*”). Consequently, the model defaults back to the statistical safety of its pre trained baseline, injecting generalized, non retrieved advice and triggering RAGAS hallucination penalties. This yields a critical actionable insight: while edge deployed quantized models enhance data sovereignty and reduce operational costs, their inability to reliably adhere to strict negative boundaries makes them inherently dangerous as autonomous advisory systems. Real world enterprise deployment targeting high faithfulness requires dedicated, high VRAM computing clusters capable of running models at uncompressed bfloat16 precision.

B. Ethical Implications and Systemic Representational Bias

Beyond computational limitations, a fundamental ethical risk of this RAG system lies in the inherent socio-economic bias of its underlying knowledge base. The system retrieves intelligence exclusively from elite, enterprise grade standards like NIST SP 800-53 [10]. As critically highlighted by Bender et al. [17] in their discourse on representational harms, algorithms often inadvertently perpetuate and amplify the structural, socio-economic biases of their retrieval corpora.

These specific cybersecurity frameworks are designed for highly mature, heavily funded corporate environments. If this RAG system is deployed in an under-resourced setting such as a rural non profit organization, a public school district, or an under funded local municipal council—the generated guidelines will be technically “accurate” against the document corpus but practically unfeasible for the end-user. For example, advising a small medical clinic to “deploy an isolated zero trust sandbox environment and trigger an automated EDR isolation playbook” provides zero actionable utility if the organization lacks a dedicated IT budget or infrastructure. By uncritically propagating enterprise centric security paradigms as universal solutions, the RAG system inadvertently exacerbates the “cybersecurity poverty line,” completely ignoring the critical need for accessible, low-maturity, and cost effective mitigation strategies [18].

C. Transparency and Governance (Human in the Loop)

To mitigate the severe risks of automated, potentially hallucinated decision making, algorithmic transparency was embedded as a core systemic requirement [19]. The strict prompt boundaries were engineered to force the LLM to explicitly append the exact source file name (e.g., `cisa_ransomware_playbook.pdf`) to every generated guideline. This deterministic citation ensures that the

provenance of the advice is entirely visible and verifiable by the end user, facilitating immediate human fact checking against the primary document.

Despite this engineered transparency, deploying this system as an autonomous incident response agent presents entirely unacceptable operational risks given the 40% hallucination rate measured during evaluation. It is strongly, unequivocally recommended that this RAG pipeline be governed strictly as an “Advisory Copilot.” A mandatory Human in the Loop (HITL) framework must be programmatically enforced, wherein all LLM generated containment strategies are flagged for critical, manual review by a human Tier-2 SOC (Security Operations Center) Analyst before any active mitigation scripts, firewall blocks, or network isolation protocols are authorized for execution on a live production network.

VI. CONCLUSION

This project successfully designed, implemented, and empirically evaluated a domain specific RAG pipeline intended for professional cybersecurity practitioners. By engineering a multi-modal ingestion strategy—combining unstructured compliance frameworks, structured CISA threat intelligence, and live MITRE ATT&CK tactical web data the system established a highly authoritative and dynamically verifiable knowledge base. The implementation of a Hybrid Retriever, merging BM25 sparse lexical search with Dense Vector semantic search, successfully ensured the precise, high recall extraction of specific technical nomenclature.

However, while strict prompt engineering over a 4-bit quantized Mistral-7B model attempted to eliminate extrinsic hallucinations, the system achieved a moderate Faithfulness score of 0.6000. Through deep evaluation and error analysis, this study revealed that aggressive NF4 quantization significantly degrades a model’s instruction following capabilities, specifically its adherence to negative prompt constraints, leading to unavoidable baseline hallucinations.

As conclusively demonstrated by the RAGAS evaluation, the edge deployment of advanced RAG systems is fundamentally bounded by severe hardware limitations, complex asynchronous software dependencies, quantization induced memory degradation, and profound socio-technical biases regarding enterprise maturity. Future iterations of this architecture must prioritize optimizing context window memory management, utilizing higher precision baseline models, and actively diversifying the knowledge base to include accessible, low-cost mitigation strategies. Resolving these computational and ethical bottlenecks is paramount to ensuring that AI driven cybersecurity guidance remains both technically unassailable and universally equitable across all organizational maturity levels.

CODE AND DATA AVAILABILITY

The complete replication package, including the deeply commented Jupyter execution notebook, multi-modal knowledge base datasets, and automated sequential evaluation scripts, is publicly available at:

https://github.com/NatarajanSellaia/assignment3_a1815352_sellaiya.git.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [2] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, “Survey of Hallucination in Natural Language Generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
- [3] C. Zhao, R. De Maria, T. Kumarage, K. S. Chaudhary, G. Agrawal, Y. Li, J. Park, Y. Deng, Y.-C. Chen, and H. Liu, “CyberBOT: Towards Reliable Cybersecurity Education via Ontology-Grounded Retrieval Augmented Generation,” *arXiv preprint arXiv:2504.00389*, 2024.
- [4] F. Blefaria, C. Cosentino, F. A. Pironti, A. Furfaro, and F. Marozzo, “CyberRAG: An Agentic RAG cyber-attack classification and reporting tool,” *arXiv preprint arXiv:2507.02424*, 2024.
- [5] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAS: Automated Evaluation of Retrieval Augmented Generation,” in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, 2024.
- [6] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier et al., “Mistral 7B,” *arXiv preprint arXiv:2310.06825*, 2023.
- [7] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale et al., “Llama 2: Open Foundation and Fine-Tuned Chat Models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [8] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, “Retrieval-Augmented Generation for Large Language Models: A Survey,” *arXiv preprint arXiv:2312.10997*, 2023.
- [9] I. Strom, “MITRE ATT&CK: Design and Philosophy,” *The MITRE Corporation*, 2018.
- [10] Joint Task Force, “Security and Privacy Controls for Information Systems and Organizations,” *National Institute of Standards and Technology (NIST) Special Publication 800-53, Revision 5*, 2020.
- [11] ISO/IEC, “Information security, cybersecurity and privacy protection — Information security management systems — Requirements,” *ISO/IEC 27001:2022*, 2022.
- [12] S. Robertson, H. Zaragoza et al., “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [13] S. Xiao, Z. Liu, P. Zhang, and N. Mu, “C-Pack: Packaged Resources To Advance General Chinese Embedding,” *arXiv preprint arXiv:2309.07597*, 2023.
- [14] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [15] L. Zheng, W.-L. Chiang, Y. Ying, S. Shen, S. Huang, I. MacY, C. H. Song, J. Wu, A. W. Riddell, H. Peng et al., “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [16] B. Wang, W. Chen, H. Pei, C. Xie, M. Kang, C. Zhang, C. Xu, Z. Xiong, R. Dutta, R. Schaeffer et al., “DecodingTrust: A Comprehensive Assessment of Trustworthiness in GPT Models,” in *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS)*, 2023.
- [17] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, “On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?,” in *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, pp. 610–623, 2021.
- [18] L. Weidinger, J. Mellor, M. Rauh, C. Griffin, J. Uesato, P.-S. Huang, M. Cheng, M. Glaese, C. Balle, A. Kasirzadeh et al., “Taxonomy of Risks posed by Language Models,” in *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency*, pp. 214–229, 2022.
- [19] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, “Concrete Problems in AI Safety,” *arXiv preprint arXiv:1606.06565*, 2016.
- [20] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray et al., “Training language models to follow instructions with human feedback,” in *Advances in Neural Information Processing Systems*, vol. 35, pp. 27730–27744, 2022.
- [21] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language Agents with Verbal Reinforcement Learning,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023.